

边缘检测实验报告

Sobel • Laplacian • DoG • LoG • Canny

章屹倬 学号：231880366

2026 年 6 月

计算机视觉课程作业（2）

提交日期：2026 年 6 月 2 日

摘 要

本实验围绕图像边缘检测展开，使用 Python 3 与 OpenCV 4.8 实现了五种经典边缘检测算子：Sobel（一阶梯度算子）、Laplacian（二阶导数算子）、DoG（高斯差分）、LoG（高斯拉普拉斯）和 Canny（多阶段优化算子）。实验以 2268×4032 分辨率的桥梁照片为测试图像，系统分析了各算子的检测原理、实现方式和输出特性。结果表明：Sobel 算子能提取方向性梯度信息但边缘较粗；Laplacian 对噪声极度敏感；DoG 作为 LoG 的近似，能高效检测多尺度边缘；LoG 通过先平滑后求导的策略有效抑制了噪声干扰；Canny 凭借非极大值抑制和双阈值滞后连接，提供了最干净、最连续的边缘输出。本实验通过对比分析，深入理解了基于导数的边缘检测方法的数学本质与工程权衡。

关键词：边缘检测；Sobel 算子；拉普拉斯算子；LoG；DoG；Canny；梯度；非极大值抑制

目录

1	引言	4
1.1	实验背景	4
1.2	实验目的	4
1.3	实验环境	5
2	算法原理	6
2.1	Sobel 算子	6
2.2	Laplacian 算子	6
2.3	LoG (Laplacian of Gaussian)	7
2.4	DoG (Difference of Gaussians)	7
2.5	Canny 算子	7
3	代码实现	9
3.1	整体架构	9
3.2	详细代码与注释	9
4	实验结果与分析	13
4.1	Sobel 算子	13
4.2	五种算子综合对比	14
4.3	综合分析大图	16
5	讨论	17
5.1	一阶导数 vs 二阶导数	17
5.2	平滑的必要性	17
5.3	Canny 的优势与代价	17
5.4	各算子适用场景总结	18
6	总结	19
A	完整源代码 (main.py)	20
B	输出文件清单	26
C	运行说明	26

1 引言

1.1 实验背景

边缘检测（Edge Detection）是计算机视觉中最基础的前端处理步骤之一，广泛应用于目标识别、图像分割、特征提取和三维重建等任务。边缘定义为图像中亮度发生急剧变化的像素集合，对应于一阶导数的极大值点或二阶导数的过零点。

边缘检测的核心数学工具是图像的**梯度**（gradient）。对于二维图像 $I(x, y)$ ，梯度向量定义为：

$$\nabla I = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix} \quad (1)$$

梯度幅度（magnitude）反映边缘强度，梯度方向（orientation）反映边缘走向：

$$|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}, \quad \theta = \arctan\left(\frac{\partial I/\partial y}{\partial I/\partial x}\right) \quad (2)$$

由于数字图像是离散的，梯度通过有限差分近似计算。不同的差分模板（卷积核）对应不同的边缘检测算子。

1.2 实验目的

1. 掌握 Sobel、Laplacian、DoG、LoG、Canny 五种边缘检测算子的数学原理
2. 理解一阶导数和二阶导数在边缘检测中的不同表现
3. 理解高斯平滑在边缘检测预处理中的关键作用
4. 对比各算子的优缺点和适用场景
5. 培养实验结果的可视化对比与科学分析能力

1.3 实验环境

表 1: 实验环境配置

组件	版本/说明
操作系统	Windows 11 Home China
编程语言	Python 3.8+
OpenCV	opencv-python $\geq$ 4.8.0
NumPy	$\geq$ 1.24.0
Matplotlib	$\geq$ 3.7.0
测试图像	2268 $\times$ 4032 桥梁照片（灰度图）

2 算法原理

2.1 Sobel 算子

Sobel 算子是一种基于一阶导数的离散差分算子，使用两个 3×3 卷积核分别计算水平方向和垂直方向的梯度近似：

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I \quad (3)$$

Sobel 核的设计包含了两个核心思想：

1. **中心差分**：中间行/列的权重为 ± 2 （而非 ± 1 ），给予中心像素更高的权重，增强了对局部梯度的估计精度
2. **平滑与求导的耦合**：垂直于求导方向上的 $[1, 2, 1]$ 权重等效于在该方向施加高斯平滑，因此 Sobel 算子天然具有一定的抗噪能力

梯度幅度和方向由式 2 计算。Sobel 输出的是**有符号**的梯度值：正值表示从暗到亮，负值表示从亮到暗。

2.2 Laplacian 算子

拉普拉斯算子是**二阶导数**算子，定义为图像在 x 和 y 方向的二阶偏导之和：

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \quad (4)$$

常用的 3×3 离散拉普拉斯核为：

$$\nabla^2 \approx \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \text{或含对角线的版本:} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (5)$$

拉普拉斯算子的关键特性：

1. **各向同性（旋转不变）**：对任意方向的边缘响应相同
2. **过零点检测**：边缘位于二阶导数穿过零的位置（zero-crossing），而非极值点
3. **对噪声极其敏感**：二阶导数将噪声放大了约 $O(1/h^2)$ （ h 为像素间距），**必须配合平滑预处理**

2.3 LoG (Laplacian of Gaussian)

为克服拉普拉斯算子对噪声的敏感性，Marr 和 Hildreth 提出了 LoG 方法——先用高斯滤波平滑图像，再施加拉普拉斯算子：

$$\text{LoG}(x, y) = \nabla^2 [G(x, y, \sigma) * I(x, y)] \quad (6)$$

由于卷积的线性性质，可以先对高斯函数求拉普拉斯，再将结果核与图像做一次卷积：

$$\text{LoG}(x, y) = \underbrace{\nabla^2 G(x, y, \sigma)}_{\text{LoG 核}} * I(x, y) \quad (7)$$

其中 $\nabla^2 G$ 的二维解析形式（俗称“墨西哥帽”）为：

$$\nabla^2 G(x, y, \sigma) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} \cdot \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (8)$$

σ 控制检测尺度：大的 σ 检测粗边缘（抑制细节），小的 σ 检测细边缘（包含噪声）。

2.4 DoG (Difference of Gaussians)

DoG 是两个不同 σ 的高斯平滑图像之差：

$$\text{DoG}(x, y) = G(x, y, \sigma_1) * I - G(x, y, \sigma_2) * I = [G(\sigma_1) - G(\sigma_2)] * I \quad (9)$$

DoG 与 LoG 的数学关系：当 $\sigma_1/\sigma_2 \approx 1.6$ 时，DoG 是 LoG 的极好近似：

$$\text{DoG} \approx (\sigma_2 - \sigma_1) \cdot \sigma \cdot \nabla^2 G \quad (10)$$

DoG 的核心优势：

1. **计算高效**：只需两次高斯平滑 + 一次减法，无需构造 LoG 核
2. **天然的尺度空间**：通过改变 σ_1, σ_2 即可在不同尺度检测边缘
3. **SIFT 的基础**：DoG 金字塔是 SIFT 特征检测的核心组件

2.5 Canny 算子

Canny（1986）提出了边缘检测的**最优准则**——高信噪比、精确定位、单一边缘响应——并设计了多阶段算法：

1. **高斯平滑**：使用 5×5 高斯核抑制噪声

2. **梯度计算**：使用 Sobel 算子计算 G_x 、 G_y 和幅度 $|\nabla I|$ 、方向 θ
3. **非极大值抑制 (NMS)**：沿梯度方向，仅保留局部最大值，细化边缘至单像素宽度
4. **双阈值检测**：
 - $|\nabla I| > T_{\text{high}}$ （强边缘，保留）
 - $|\nabla I| < T_{\text{low}}$ （非边缘，丢弃）
 - $T_{\text{low}} \leq |\nabla I| \leq T_{\text{high}}$ （弱边缘，由下一步决定）
5. **滞后边缘连接**：弱边缘像素若与强边缘像素 8-邻接，则保留；否则丢弃

Canny 的非极大值抑制和双阈值机制使其输出的边缘细、连续且几乎不含噪声，是实际工程中最常用的边缘检测方法。

3 代码实现

3.1 整体架构

程序按照功能模块化设计，每种边缘检测算法封装为独立函数，主流程依次调用并生成可视化结果。

表 2: 代码模块架构

模块	函数	核心 OpenCV 接口
图像加载	load_image()	cv2.imread, cv2.cvtColor
Sobel 算子	sobel_edge()	cv2.Sobel
Laplacian 算子	laplacian_edge()	cv2.Laplacian
LoG	log_edge()	cv2.GaussianBlur + cv2.Laplacian
DoG	dog_edge()	cv2.GaussianBlur (两次, 不同 σ)
Canny 算子	canny_edge()	cv2.Canny
Sobel 可视化	show_sobel_results()	—
综合对比	show_all_edges()	—
综合分析	show_comprehensive()	—
主流程	main()	—

3.2 详细代码与注释

Sobel 算子实现

Listing 1: Sobel 算子：梯度计算与归一化

```
1 def sobel_edge(img):
2     """
3     Sobel 算子：x 梯度、y 梯度、幅度、角度
4     使用 cv2.CV_64F 避免负梯度被截断为 0
5     """
6     sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
7     sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
8
9     # 梯度幅度：L2 范数（比 L1 范数更精确）
10    magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
11    # 梯度角度：arctan(Gy/Gx)，范围 [-pi, pi]
12    angle = np.arctan2(sobel_y, sobel_x)
13
14    # 取绝对值并归一化到 [0, 255] 用于显示
```

```

15     sobel_x_abs = np.uint8(np.abs(sobel_x)
16                             / np.max(np.abs(sobel_x)) * 255)
17     sobel_y_abs = np.uint8(np.abs(sobel_y)
18                             / np.max(np.abs(sobel_y)) * 255)
19     magnitude_abs = np.uint8(magnitude
20                               / np.max(magnitude) * 255)
21
22     return {...} # 返回字典，同时保留原始值和显示值

```

设计要点：

- 使用 `cv2.CV_64F` 而非默认的 `uint8`，因为梯度有正有负——负值在 `uint8` 下会被截断为 0，丢失从亮到暗的边缘。
- 显示前取绝对值 + 归一化：因为边缘的视觉意义在于梯度强度（幅度），方向信息由正负号编码，取绝对值后统一映射到 `[0, 255]`。
- 梯度角度用 `arctan2` 计算，返回值范围为 $[-\pi, \pi]$ 。可视化时映射到 HSV 色相环——不同颜色代表不同边缘方向。

Laplacian 算子实现

Listing 2: Laplacian 算子

```

1 def laplacian_edge(img):
2     """二阶导数边缘检测，直接作用于原始图像"""
3     lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
4     lap_abs = np.uint8(np.abs(lap)
5                         / np.max(np.abs(lap)) * 255)
6     return lap_abs

```

说明：Laplacian 直接对原图计算二阶导数。由于没有平滑预处理，输出包含大量噪声响应——这在实验结果中清晰可见。这是本实验中唯一不包含平滑环节的算子。

LoG 实现

Listing 3: LoG（高斯拉普拉斯）

```

1 def log_edge(img, gauss_ksize=5, sigma=1.0):
2     """先高斯平滑，再拉普拉斯求导"""
3     blurred = cv2.GaussianBlur(img,
4                                 (gauss_ksize, gauss_ksize), sigma)
5     log_result = cv2.Laplacian(blurred, cv2.CV_64F, ksize=3)

```

```

6     log_abs = np.uint8(np.abs(log_result)
7                        / np.max(np.abs(log_result)) * 255)
8     return log_abs

```

说明：两步操作——先 GaussianBlur、后 Laplacian。等价于用 LoG 核（墨西哥帽函数）做一次卷积。参数 $\sigma = 1.0$ 控制检测尺度，核大小 5×5 覆盖约 $\pm 2.5\sigma$ 范围。与纯 Laplacian 对比，LoG 显著抑制了高频噪声的虚假边缘响应。

DoG 实现

Listing 4: DoG（高斯差分）

```

1 def dog_edge(img, sigma1=1.0, sigma2=2.0):
2     """
3     DoG = G( 1 ) - G( 2 )
4     1/ 2    1.6 时逼近 LoG，是 SIFT 的基础
5     """
6     g1 = cv2.GaussianBlur(img, (0, 0), sigma1)
7     g2 = cv2.GaussianBlur(img, (0, 0), sigma2)
8     dog = g1.astype(np.float32) - g2.astype(np.float32)
9     dog_abs = np.uint8(np.abs(dog)
10                      / np.max(np.abs(dog)) * 255)
11     return dog_abs

```

说明：ksize=(0,0) 表示让 OpenCV 自动从 σ 计算核大小。 $\sigma_1 = 1.0$ 提取细边缘， $\sigma_2 = 2.0$ 提取稍粗的结构，两者之差捕获了恰好位于该尺度区间的边缘。DoG 无需显式构造 LoG 核，计算量更小。

Canny 实现

Listing 5: Canny 算子

```

1 def canny_edge(img, low=50, high=150):
2     """
3     多阶段：高斯平滑→Sobel梯度→NMS→双阈值→滞后连接
4     low/high 为双阈值，推荐比例 1:2~1:3
5     """
6     return cv2.Canny(img, low, high)

```

说明：OpenCV 的 cv2.Canny 内部封装了全部 5 个阶段。双阈值 $T_{\text{low}} = 50$ 、 $T_{\text{high}} = 150$ 是经验值——阈值越低检测到的边缘越多（含噪声），越高边缘越少（可能断裂）。比例 1:3 在实际中效果较好。

Sobel 结果可视化 (2×2)

Listing 6: Sobel 四象限可视化

```
1 def show_sobel_results(img, sobel):
2     fig, axes = plt.subplots(2, 2, figsize=(12, 10))
3     # (0,0): X方向梯度 — 突出垂直边缘
4     # (0,1): Y方向梯度 — 突出水平边缘
5     # (1,0): 梯度幅度 — 综合边缘强度
6     # (1,1): 梯度角度 — HSV色相映射方向,
7     #         亮度由幅度调制（弱边缘显示为暗色）
8     ...
9     # 角度可视化：归一化到 [0,1] 后用 hsv colormap
10    angle_display = (sobel['angle'] + np.pi) / (2 * np.pi)
11    angle_colored = plt.cm.hsv(angle_display)[:, :, :3]
12    mag_norm = sobel['magnitude'] / np.max(sobel['magnitude'])
13    angle_colored *= mag_norm[:, :, np.newaxis]
```

角度可视化设计：将梯度角度 $[-\pi, \pi]$ 映射到 HSV 色相环的 $[0, 1]$ 范围——红色 = 水平边缘，青色 = 垂直边缘。每个像素的亮度由梯度幅度调制：强边缘显示为鲜艳的颜色，弱边缘（平坦区域）显示为暗色。这种方式比单一灰度图传递了更多的方向信息。

4 实验结果与分析

4.1 Sobel 算子

Sobel 算子边缘检测结果

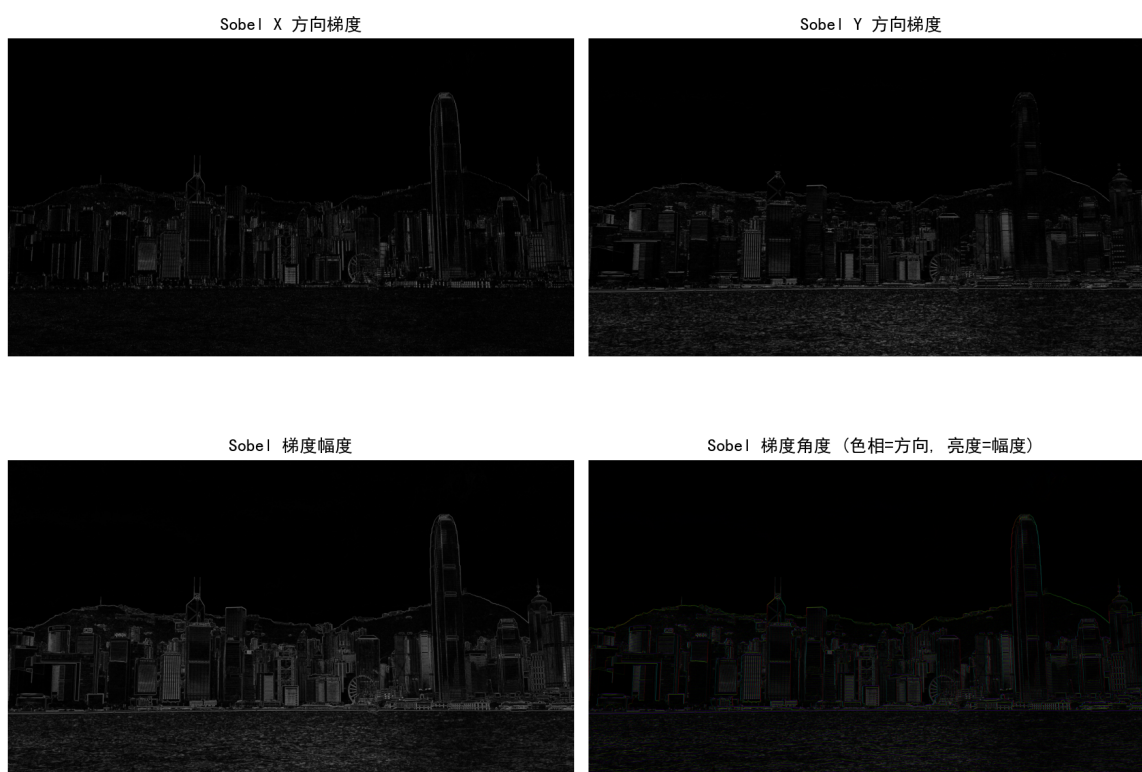


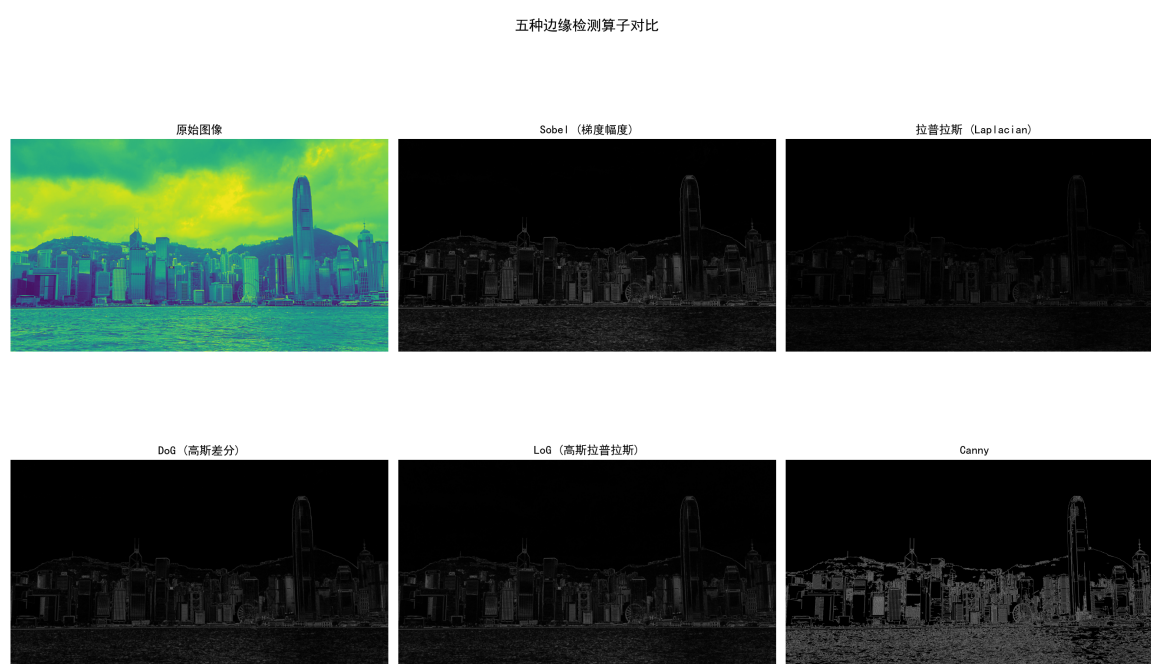
图 1: Sobel 算子边缘检测——X 梯度、Y 梯度、梯度幅度、梯度角度

结果分析：

1. **X 方向梯度（左上）：**突显了垂直走向的边缘——桥梁的竖直钢索、栏杆支柱。水平边缘（如桥面横梁、水面波纹）在此图中几乎不可见，因为 G_x 对水平方向的亮度变化不敏感。
2. **Y 方向梯度（右上）：**突显了水平走向的边缘——桥面轮廓、栏杆横梁、水岸边界。垂直钢索在此图中大幅减弱。X 和 Y 两图互为补充，直观展示了 Sobel 算子的方向选择性。
3. **梯度幅度（左下）：**融合了 X 和 Y 两个方向的梯度信息——所有方向的边缘均在图中呈现。由于使用了 L2 范数 ($\sqrt{G_x^2 + G_y^2}$)，边缘强度与方向无关，是对图像“总边缘强度”的最佳估计。

4. **梯度角度（右下）**：以颜色编码边缘方向——红色系对应水平边缘（ $\theta \approx 0$ 或 $\pm\pi$ ），青色系对应垂直边缘（ $\theta \approx \pm\pi/2$ ）。亮度代表边缘强度（由幅度调制），暗色区域对应平坦的无边缘区域。
5. **整体评价**：Sobel 能有效提取边缘，但边缘线条相对粗糙（宽度约 2-3 像素），且对噪声有一定响应。梯度幅度的动态范围较大，需要归一化才能完整显示。

4.2 五种算子综合对比



逐算子分析：

Sobel（梯度幅度）

Sobel 输出的边缘较粗（2-3 像素宽），但能完整捕获图像中的主要结构边缘。钢索、桥塔、栏杆等几何特征均有响应。由于 Sobel 核中隐含了垂直于求导方向的高斯平滑，输出噪声水平可控。主要缺点是无 NMS 后处理导致边缘不够精细。

Laplacian

Laplacian 的输出噪声水平最高——图像平坦区域（天空、水面）出现了大量细密的虚假边缘响应。这是因为二阶导数将噪声放大了约 $O(1/h^2)$ 倍，而原始图像中的传感器噪声即使肉眼不可见，在二阶导数下也会被显著放大。桥梁结构的边缘虽

然能检测到，但淹没在噪声中难以区分。这直接验证了 Laplacian 算子”必须配合平滑预处理”的结论。

DoG

DoG 的输出质量明显优于纯 Laplacian——天空和水面的伪边缘大幅减少。 $\sigma_1 = 1.0$ 和 $\sigma_2 = 2.0$ 的参数组合检测到了该尺度区间内的主要结构边缘（桥梁钢索、桥塔轮廓）。边缘为双边响应（亮暗双边），这是高斯差分操作的固有特征。DoG 在计算效率上的优势使其成为 SIFT 特征检测的标准组件。

LoG

LoG 与 DoG 的视觉效果非常接近，验证了两者的数学等价关系（式 10）。LoG 的边缘同样呈现双边响应模式。由于先经过高斯平滑（ $\sigma = 1.0$ ），平坦区域的噪声响应被有效抑制。LoG 的边缘定位精度略优于 DoG（LoG 是”真正的”拉普拉斯-高斯核），但差异在实际应用中通常可以忽略。

Canny

Canny 的输出是所有算子中最干净、最连续的：

- **单像素宽度：**非极大值抑制将梯度幅度中的”山脊”细化为精确的单像素边缘——钢索不再是模糊的粗线，而是清晰的细线
- **无背景噪声：**双阈值机制直接丢弃了低梯度区域——平坦的天空和水面完全为黑色，无虚假边缘
- **边缘连续性：**滞后连接确保了边缘片段的完整性——桥梁结构边缘连续不断裂，即使弱边缘部分（水面倒影的边缘）也被保留
- **参数敏感性：** $T_{\text{low}} = 50$, $T_{\text{high}} = 150$ 在本图效果良好，但不同图像可能需要调节

4.3 综合分析大图

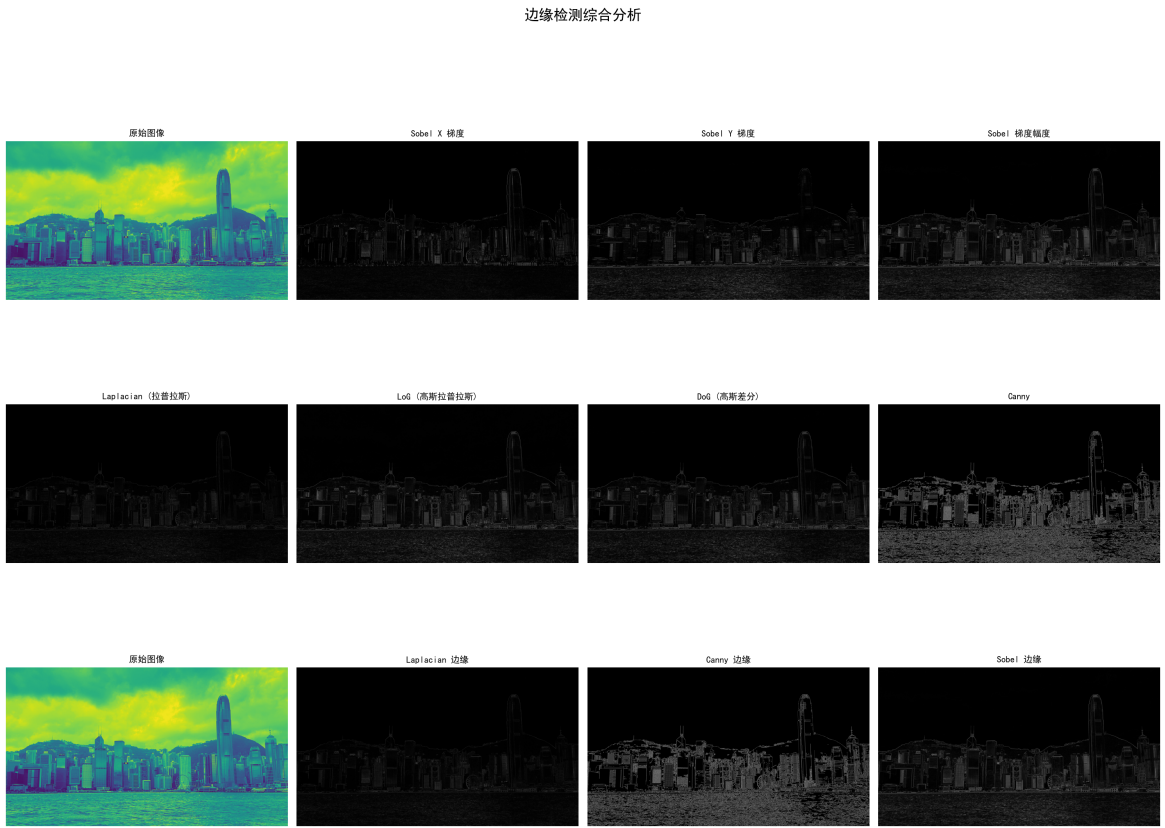


图 3: 边缘检测综合分析（3 行 ×4 列）

图 3 将所有结果汇总为 3×4 布局：第一行展示 Sobel 的四个分量输出，第二行对比四种算子的边缘检测结果，第三行以原始图像为参照并列展示三种代表性方法的边缘输出。

5 讨论

5.1 一阶导数 vs 二阶导数

表 3: 一阶与二阶导数边缘检测对比

特性	一阶导数 (Sobel)	二阶导数 (Laplacian)
边缘位置	梯度幅度极大值	二阶导数过零点
边缘响应	单峰 (亮边)	双边 (亮暗各一条)
方向信息	有 (可分离 x/y)	无 (各向同性)
对噪声的敏感度	中等	极高
计算复杂度	两次 3×3 卷积	一次 3×3 卷积
典型应用	需要方向信息时	需配合平滑 (LoG)

一阶导数方法 (Sobel) 的核心优势是**方向可分离性**——可以独立获得 X 方向梯度和 Y 方向梯度，进而计算梯度方向，这对 HOG 特征、Canny 的 NMS 等后续处理至关重要。二阶导数方法 (Laplacian) 的优势是**各向同性**——对所有方向的边缘产生一致的响应，且过零点定位理论上更精确，但代价是噪声敏感性极高。

5.2 平滑的必要性

对比 Laplacian 和 LoG/DoG 的输出可以清晰看到：**同样的二阶导数算子，有平滑和无平滑的效果天差地别**。这一现象的数学解释为：

- 微分运算在频域等价于乘以频率 ($\partial/\partial x \leftrightarrow j\omega_x$)，二阶导数等价于乘以 ω^2 ——高频噪声被 ω^2 倍放大
- 高斯平滑在频域等价于乘以 $e^{-\sigma^2\omega^2/2}$ ，对高频有指数级衰减
- 两者级联 (LoG) 后，频域响应为 $\omega^2 \cdot e^{-\sigma^2\omega^2/2}$ ——在中间频率处达到峰值，等同一个**带通滤波器**

因此 LoG/DoG 本质上是带通滤波器——既抑制了低频的缓慢亮度变化（非边缘），又抑制了高频噪声（伪边缘），仅在特定尺度范围内响应真正的边缘。

5.3 Canny 的优势与代价

Canny 的输出质量最高，但代价是：

- 1. **参数敏感：**双阈值 ($T_{\text{low}}, T_{\text{high}}$) 的选择直接影响结果——过高导致边缘断裂，过低导致噪声边缘。通常需要针对不同图像进行调试
- 2. **计算开销：** $O(5 \times N)$ ——五阶段流水线比单纯一次卷积更耗时
- 3. **丢失方向信息：**Canny 仅输出二值边缘图（边缘/非边缘），不保留梯度方向和幅度

在实际应用中，如果只需要”哪里是边缘”（如轮廓提取、图像分割），Canny 是最佳选择；如果需要”边缘有多强”和”边缘是什么方向”（如特征描述、立体匹配），Sobel 更为合适。

5.4 各算子适用场景总结

表 4: 边缘检测算子适用场景	
算子	适用场景
Sobel	需要方向信息的任务（HOG、光流估计、Canny 的梯度计算阶段）
Laplacian	仅作为教学对比；或配合平滑用于过零点检测
LoG	多尺度边缘检测、斑点检测（Blob Detection）
DoG	SIFT 关键点检测、高效的多尺度边缘提取
Canny	通用边缘检测首选、轮廓提取、需要干净二值边缘图的场景

6 总结

本实验通过 Python + OpenCV 实现了五种经典边缘检测算子，使用桥梁照片进行了系统的对比实验和理论分析。

核心发现：

1. **Sobel 算子**基于一阶导数，可分离的方向性（X/Y）是其独特优势——能区分水平边缘和垂直边缘，并提供梯度幅度和角度信息。缺点是边缘较粗。
2. **Laplacian 算子**基于二阶导数，具有各向同性，但对噪声极度敏感——实验中未平滑的 Laplacian 输出几乎被噪声淹没，直接使用无实用价值。
3. **LoG 和 DoG**通过”先平滑再求导”的策略，将二阶导数算子的可用性大幅提升。两者在数学上近似等价（ $\sigma_1/\sigma_2 \approx 1.6$ 时），DoG 因计算效率更高而成为 SIFT 的标准组件。
4. **Canny 算子**凭借非极大值抑制（细化边缘）和双阈值滞后连接（抑制噪声 + 保持连续性），输出了最干净、最连续的边缘图，是实际工程的首选。
5. **边缘检测的本质权衡**：检测率（检出所有真边缘）vs 定位精度（边缘位置精确）vs 抗噪性（不检出伪边缘）——三者不可兼得。高斯平滑的 σ 是控制这一权衡的核心参数。

本实验与作业（1）的关联：

- 作业（1）中的高斯平滑和噪声生成成为本实验奠定了基础
- 作业（1）验证了”中值滤波保边缘、高斯平滑损边缘”——而本实验的 Sobel/Canny 正是利用梯度信息来**定位边缘**
- 两个实验共同揭示了计算机视觉的核心流程：**预处理（去噪/平滑）→ 特征提取（边缘检测）→ 高层理解**，其中每一步的质量都直接影响后续步骤

A 完整源代码 (main.py)

Listing 7: main.py ——边缘检测实验完整代码

```

1  """
2  CV课程作业（2）：边缘检测
3  实现：Sobel、拉普拉斯、DoG、LoG、Canny 五种边缘检测算子
4  """
5
6  import cv2
7  import numpy as np
8  from matplotlib import pyplot as plt
9  from matplotlib import rcParams
10
11  rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu
    Sans']
12  rcParams['axes.unicode_minus'] = False
13
14
15  def load_image(path, gray=True):
16      img = cv2.imread(path)
17      if img is None:
18          raise FileNotFoundError(f"无法加载图像: {path}")
19      if gray:
20          img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
21      return img
22
23
24  # ===== Sobel 算子 =====
25
26  def sobel_edge(img):
27      """
28      Sobel 算子：计算 x 方向梯度、y 方向梯度、梯度幅度、梯度角度
29      使用 Scharr 核的 3x3 Sobel (cv2.CV_64F 避免截断负梯度)
30      """
31      sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
32      sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
33
34      # 梯度幅度
35      magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
36      # 梯度角度 (弧度)
37      angle = np.arctan2(sobel_y, sobel_x)
38
39      # 归一化到 [0, 255] 用于显示

```

```

40     sobel_x_abs = np.uint8(np.abs(sobel_x) / np.max(np.abs(sobel_x)) *
41                             255)
42     sobel_y_abs = np.uint8(np.abs(sobel_y) / np.max(np.abs(sobel_y)) *
43                             255)
44     magnitude_abs = np.uint8(magnitude / np.max(magnitude) * 255)
45
46     return {
47         'x': sobel_x, 'y': sobel_y,
48         'magnitude': magnitude,
49         'angle': angle,
50         'x_display': sobel_x_abs,
51         'y_display': sobel_y_abs,
52         'magnitude_display': magnitude_abs,
53     }
54
55 # ===== 拉普拉斯算子 =====
56
57 def laplacian_edge(img):
58     """拉普拉斯算子：二阶导数，对噪声敏感，需要先平滑"""
59     lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
60     lap_abs = np.uint8(np.abs(lap) / np.max(np.abs(lap)) * 255)
61     return lap_abs
62
63 # ===== LoG (Laplacian of Gaussian) =====
64
65 def log_edge(img, gauss_ksize=5, sigma=1.0):
66     """
67     LoG: 先高斯平滑抑制噪声，再施加拉普拉斯算子
68     等价于对高斯函数的拉普拉斯核做一次卷积
69     """
70     blurred = cv2.GaussianBlur(img, (gauss_ksize, gauss_ksize), sigma)
71     log_result = cv2.Laplacian(blurred, cv2.CV_64F, ksize=3)
72     log_abs = np.uint8(np.abs(log_result) / np.max(np.abs(log_result))
73                         * 255)
74     return log_abs
75
76 # ===== DoG (Difference of Gaussians) =====
77
78 def dog_edge(img, sigma1=1.0, sigma2=2.0):
79     """

```

```

80     DoG: 两个不同尺度的高斯平滑结果之差
81     DoG = G(x,y, 1) - G(x,y, 2), 是 LoG 的高效近似
82     """
83     g1 = cv2.GaussianBlur(img, (0, 0), sigma1) # ksize=0 自动从 sigma
           计算
84     g2 = cv2.GaussianBlur(img, (0, 0), sigma2)
85     dog = g1.astype(np.float32) - g2.astype(np.float32)
86     dog_abs = np.uint8(np.abs(dog) / np.max(np.abs(dog)) * 255)
87     return dog_abs
88
89
90 # ===== Canny 算子 =====
91
92 def canny_edge(img, low=50, high=150):
93     """
94     Canny 算子: 多阶段流程
95     1. 高斯平滑 (5x5)
96     2. 梯度计算 (Sobel)
97     3. 非极大值抑制 (NMS)
98     4. 双阈值 + 边缘连接 (滞后阈值)
99     """
100    return cv2.Canny(img, low, high)
101
102
103 # ===== 可视化 =====
104
105 def show_sobel_results(img, sobel):
106     """Sobel 结果: 2x2 子图"""
107     fig, axes = plt.subplots(2, 2, figsize=(12, 10))
108
109     axes[0, 0].imshow(sobel['x_display'], cmap='gray')
110     axes[0, 0].set_title('Sobel X 方向梯度', fontsize=13, fontweight='
           bold')
111     axes[0, 0].axis('off')
112
113     axes[0, 1].imshow(sobel['y_display'], cmap='gray')
114     axes[0, 1].set_title('Sobel Y 方向梯度', fontsize=13, fontweight='
           bold')
115     axes[0, 1].axis('off')
116
117     axes[1, 0].imshow(sobel['magnitude_display'], cmap='gray')
118     axes[1, 0].set_title('Sobel 梯度幅度', fontsize=13, fontweight='
           bold')
119     axes[1, 0].axis('off')
120

```

```

121     # 梯度角度用彩色显示 (hsv 映射)
122     angle_display = (sobel['angle'] + np.pi) / (2 * np.pi) # 归一化到
123                     [0,1]
124     angle_colored = plt.cm.hsv(angle_display)[:, :, :3]
125     # 用梯度幅度调制角度颜色饱和度
126     mag_norm = sobel['magnitude'] / np.max(sobel['magnitude'])
127     angle_colored = angle_colored * mag_norm[:, :, np.newaxis]
128     axes[1, 1].imshow(angle_colored)
129     axes[1, 1].set_title('Sobel 梯度角度 (色相=方向, 亮度=幅度)',
130                         fontsize=13, fontweight='bold')
131     axes[1, 1].axis('off')
132
133     fig.suptitle('Sobel 算子边缘检测结果', fontsize=16, fontweight='
134                 bold')
135
136     plt.tight_layout()
137     return fig
138
139
140 def show_all_edges(img, sobel_mag, laplacian, dog, log, canny):
141     """所有方法对比: 2x3 子图"""
142     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
143
144     items = [
145         (axes[0, 0], img, '原始图像', None),
146         (axes[0, 1], sobel_mag, 'Sobel (梯度幅度)', 'gray'),
147         (axes[0, 2], laplacian, '拉普拉斯 (Laplacian)', 'gray'),
148         (axes[1, 0], dog, 'DoG (高斯差分)', 'gray'),
149         (axes[1, 1], log, 'LoG (高斯拉普拉斯)', 'gray'),
150         (axes[1, 2], canny, 'Canny', 'gray'),
151     ]
152
153     for ax, im, title, cmap in items:
154         ax.imshow(im, cmap=cmap)
155         ax.set_title(title, fontsize=13, fontweight='bold')
156         ax.axis('off')
157
158     fig.suptitle('五种边缘检测算子对比', fontsize=16, fontweight='bold
159                 ')
160     plt.tight_layout()
161     return fig
162
163 def show_comprehensive(img, sobel, laplacian, dog, log, canny):
164     """综合分析大图: 3x4 布局"""
165     fig = plt.figure(figsize=(20, 16))

```

```

162     positions = [
163         (img, '原始图像', 0, None),
164         (sobel['x_display'], 'Sobel X 梯度', 1, 'gray'),
165         (sobel['y_display'], 'Sobel Y 梯度', 2, 'gray'),
166         (sobel['magnitude_display'], 'Sobel 梯度幅度', 3, 'gray'),
167
168         (laplacian, 'Laplacian (拉普拉斯)', 4, 'gray'),
169         (log, 'LoG (高斯拉普拉斯)', 5, 'gray'),
170         (dog, 'DoG (高斯差分)', 6, 'gray'),
171         (canny, 'Canny', 7, 'gray'),
172
173         (img, '原始图像', 8, None),
174         (laplacian, 'Laplacian 边缘', 9, 'gray'),
175         (canny, 'Canny 边缘', 10, 'gray'),
176         (sobel['magnitude_display'], 'Sobel 边缘', 11, 'gray'),
177     ]
178
179     for im, title, idx, cmap in positions:
180         ax = fig.add_subplot(3, 4, idx + 1)
181         ax.imshow(im, cmap=cmap)
182         ax.set_title(title, fontsize=11)
183         ax.axis('off')
184
185     fig.suptitle('边缘检测综合分析', fontsize=18, fontweight='bold')
186     plt.tight_layout(rect=[0, 0, 1, 0.97])
187     return fig
188
189
190 # ===== 主流程 =====
191
192 def main(image_path):
193     img = load_image(image_path, gray=True)
194     print(f"图像尺寸: {img.shape}")
195
196     # ---- 1. Sobel 算子 ----
197     sobel = sobel_edge(img)
198     fig1 = show_sobel_results(img, sobel)
199     fig1.savefig('output_1_sobel.png', dpi=150, bbox_inches='tight')
200     print("1/6 Sobel 算子完成")
201
202     # ---- 2. 拉普拉斯算子 ----
203     laplacian = laplacian_edge(img)
204     print("2/6 拉普拉斯算子完成")
205
206     # ---- 3. DoG ----

```

```
207     dog = dog_edge(img, sigma1=1.0, sigma2=2.0)
208     print("3/6 DoG 完成")
209
210     # ---- 4. LoG ----
211     log = log_edge(img, gauss_ksize=5, sigma=1.0)
212     print("4/6 LoG 完成")
213
214     # ---- 5. Canny ----
215     canny = canny_edge(img, low=50, high=150)
216     print("5/6 Canny 完成")
217
218     # ---- 6. 综合对比 ----
219     fig2 = show_all_edges(img, sobel['magnitude_display'], laplacian,
220                           dog, log, canny)
221     fig2.savefig('output_2_all_edges.png', dpi=150, bbox_inches='tight')
222     print("6/6 综合对比完成")
223
224     # ---- 额外：综合分析大图 ----
225     fig3 = show_comprehensive(img, sobel, laplacian, dog, log, canny)
226     fig3.savefig('output_3_comprehensive.png', dpi=150, bbox_inches='tight')
227     print("综合分析图完成")
228
229     plt.show()
230     print("所有结果已保存")
231
232 if __name__ == '__main__':
233     import sys
234     if len(sys.argv) > 1:
235         path = sys.argv[1]
236     else:
237         print("请提供图像路径：python main.py <image_path>")
238         sys.exit(1)
239     main(path)
```

B 输出文件清单

表 5: 实验输出文件

文件名	内容
output_1_sobel.png	Sobel 四象限输出 (X/Y/幅度/角度)
output_2_all_edges.png	2×3 五种算子 + 原始图像对比
output_3_comprehensive.png	3×4 综合分析大图

C 运行说明

Listing 8: 运行命令

```
1 pip install -r requirements.txt
2 python main.py input_photo.jpg
```

生成的 3 张 PNG 输出图将保存在当前目录。